

\$ W5-1 · FREE5GC



整體架構、原始碼倉庫與 NF 程式結構：先畫一張讀 code 的地圖

/> 課程大綱

00

RECAP

5GC 架構快速回顧

01

定位

free5GC 是什麼、適合做什麼

02

比較

開源 5GC 與 Go 語言

03

架構

實驗拓樸與部署型態

04

倉庫

主倉庫、NFs 與函式庫

05

NF 分類

五大類網路功能

06

程式結構

cmd / internal / pkg

07

讀 CODE

追一個 SBI 請求

\$ RECAP · W3

➤ 5GC 架構回顧

把上次的 3GPP 大圖帶進今天的開源實作

➤ SBA 與 NF：上次的重點

5GC 由多個 NF 組成，NF 之間透過 SBI (HTTP/2 + JSON) 互相呼叫

AMF 管註冊與行動性、SMF 管 Session、UPF 搬使用者資料

NRF 是 NF 的「黃頁」，負責服務註冊與發現

UDM / UDR / AUSF 管身分與訂閱，PCF 管策略，NSSF 管切片

／> 接口地圖 → 今天的問題

控制面

N1 · N2 · N4 · Nn

N1 NAS、N2 NGAP

N4 PFCP：SMF 指揮 UPF

Nn：NF 之間的 SBI

使用者面

N3 · N6

N3 GTP-U：gNB ↔ UPF

N6：UPF ↔ DN

真正的封包在這裡流動

今天的問題：這些「規格上的方塊」，在 free5GC 裡變成了哪些倉庫、哪些程式？

\$ PART 01

FREE5GC 是什麼？

一個以教學、研究與 PoC 為主的開源 5G Core

➤ FREE5GC 的定位

開源 5G Core 專案，目標是 教學、研究與 PoC

實作 5GC 控制面與使用者面，以 3GPP R15 為基礎，現已升級到 R17

可跑在一般伺服器、虛擬機或容器中，快速建出實驗平台

重視可讀性、可修改性、易於部署，而非電信等級的完整度

官方網站：free5gc.org | 原始碼：github.com/free5gc/free5gc

／> 四個關鍵字認識 FREE5GC

ORIGIN

學術出身

由台灣陽明交通大學團隊主導開發，社群持續維護

LANG

Go 語言

所有 NF 以 Go 撰寫，易讀、並行能力佳

SPEC

3GPP R15 → R17

跟著標準演進，持續補上新版本功能

LICENSE

Apache 2.0

寬鬆授權，可修改、可再利用、可整合進產品

／> 適合用在哪裡？

A

研究與教學

學術或企業內部 5G 課程、論文實驗平台

B

概念驗證

驗證新協定、新演算法、新架構的 PoC

C

E2E 實驗

搭配開源 RAN / UE，打通端到端連線

例：修改 SMF 的 UPF 選擇策略，觀察延遲變化

例：在 AMF 加 log，追蹤一次 Registration 的完整訊令

／> 它不是什麼？

不追求完整覆蓋所有 3GPP 功能，部分流程仍在補齊中

沒有電信等級的 HA（高可用）、大規模擴充設計

不建議直接用於商用生產環境

本課程的角度：把它當成「可以打開來讀的 5GC 教材」，而非黑盒子

➤ 開源 5GC 專案比較

專案	語言	特色
free5GC	Go	完整遵循 SBA、每個 NF 獨立倉庫、教學導向
Open5GS	C	同時支援 4G EPC 與 5GC，輕量、社群活躍
OAI CN5G	C++	OpenAirInterface 生態，常與 OAI RAN 搭配

沒有絕對的好壞：選擇取決於研究目標、熟悉的語言，以及要搭配的 RAN。

／> 為什麼這門課選 FREE5GC ?

01

GO 好上手

語法簡潔、工具鏈完整，並行模型適合大量訊令

02

SBA 完整

嚴格依 3GPP SBA 拆分，模組化、易擴展

03

教學導向

文件與範例環境友善，可一步步跟著實作

學會讀 free5GC，就等於學會把 3GPP 規格「對應到程式碼」的方法。

➤ GO 與 5GC 的契合點

GOROUTINE

輕量並行

每個 UE、每個請求都能用 goroutine 處理

NET/HTTP

原生 HTTP/2

SBI 需要的 HTTP/2 與 JSON 標準庫就有

BINARY

單一執行檔

靜態編譯，部署到 VM / 容器都很簡單

TOOLING

工具鏈一致

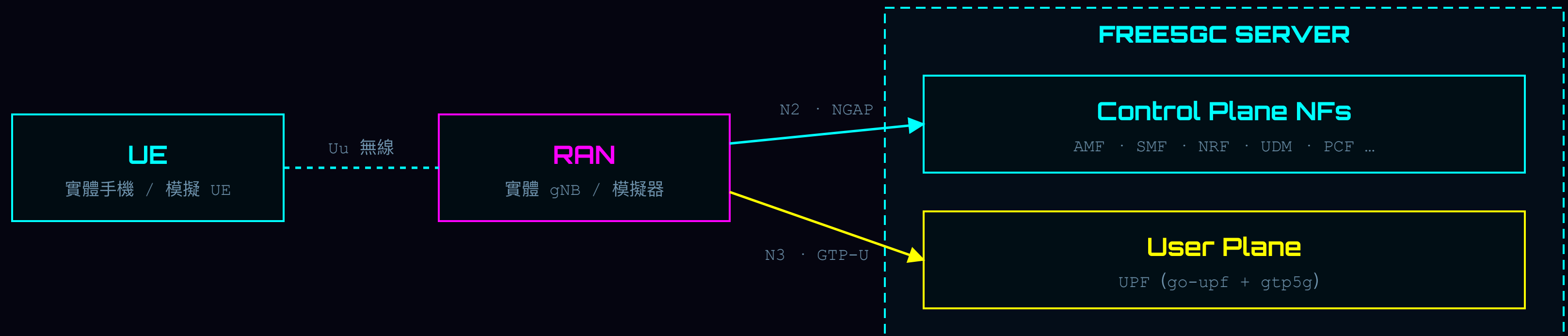
go mod、go test、gofmt 讓多倉庫風格統一

\$ PART 02

FREE5GC 架構

實驗拓樸、部署型態與 Github 生態系

/> 典型實驗拓樸



模擬 UE / RAN 常見選擇：UERANSIM、PacketRusher、free-ran-ue；也可接實體 gNB 與 SIM 卡手機。

／> 三種部署型態

單機

All-in-one

所有 NF + DB 同一台

快速體驗、好除錯

課堂實驗首選

多節點

Multi-node

DB、UPF 拆出去

貼近實務拓樸

可做 UPF 邊緣部署

容器化

Docker / K8s

free5gc-compose

free5gc-helm

易整合、上雲測試

詳細部署步驟會在之後的「部署」單元介紹，今天先建立整體概念。

➤ GITHUB 生態系：不只一個倉庫

主倉庫

`free5gc/free5gc`

NFs/：以 submodule 引入各 NF

config/：各 NF 的 YAML 設定檔

Makefile、run.sh：建置與啟動

test/：整合測試

周邊倉庫

`free5gc/<repo>`

每個 NF 一個獨立倉庫

共用的協定與工具函式庫

webconsole、部署專案

各自版本、各自發 PR

主倉庫像「組裝線」：把各 NF 版本釘在一起，確保整套能一起建置與運作。

➤ 主倉庫長什麼樣子

```
free5gc/
├── NFs/                # submodules
│   ├── amf/           ├── smf/
│   ├── upf/           ├── nrf/
│   └── ...
├── config/             # *cfg.yaml
├── webconsole/
├── test/
├── Makefile
└── run.sh
```

`git clone --recursive` 一次拉下所有 NF

`make` 編出各 NF 執行檔到 bin/

`run.sh` 依序啟動整套核心網

➤ NFS / 目錄裡有誰？

AMF

接入與行動性

SMF

Session 管理

UPF

使用者面轉送

NRF

註冊與發現

AUSF

認證服務

UDM

訂閱資料管理

UDR

資料儲存庫

PCF

策略控制

CHF

計費

NSSF

切片選擇

NEF

能力開放

N3IWF

非 3GPP 接入

另有 TNGF（可信任非 3GPP 接入）與 WebConsole（free5GC 自製網管，非 3GPP NF）。

➤ 共用函式庫：NF 背後的積木

openapi 依 3GPP OpenAPI 產生的 SBI 資料模型與 client

ngap / aper N2 的 NGAP 訊息與 ASN.1 APER 編解碼

nas N1 NAS 訊息編解碼與安全保護

pfcf N4 PFCP 協定：SMF 指揮 UPF

gtp5g Linux kernel module，UPF 的 GTP-U 資料面

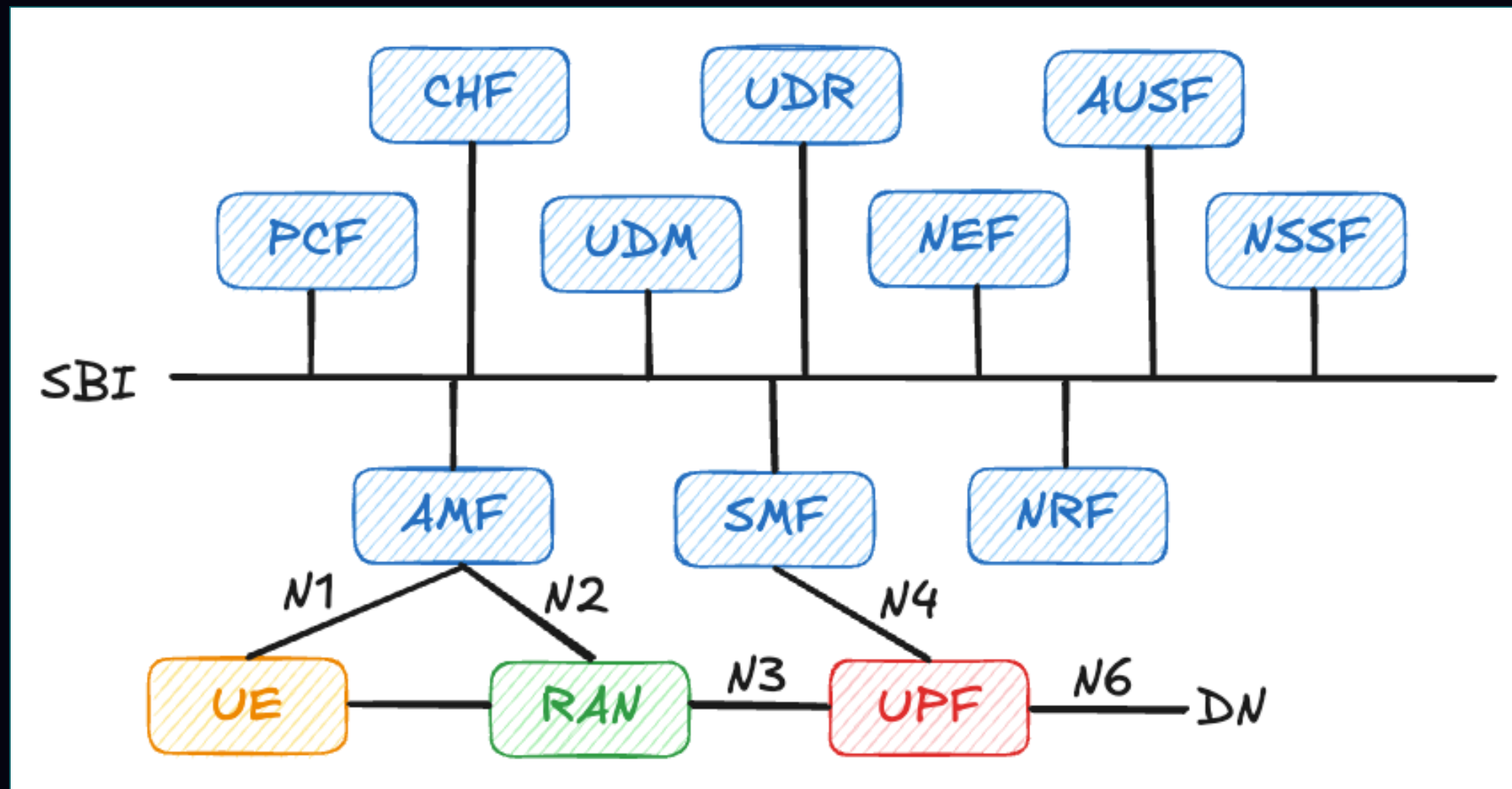
util 共用工具：log、mongo 存取、HTTP 包裝等

\$ PART 03

➤ 網路元件模組

依 3GPP 5GC 定義，把 NF 放回 SBA 的大圖中

➤ FREE5GC 的 SBA 全貌



上半部：掛在 SBI 匯流排上的 NF

下半部：N1~N6 點對點接口

UPF 不走 SBI，靠 N4 受 SMF 控制

➤ 五大類網路功能

ACCESS & MOBILITY

AMF

UE 註冊、連線與行動性管理，進入 5GC 的入口

SESSION & UP

SMF / UPF

PDU Session 建立、修改、釋放與資料轉送

AUTH & SUBSCRIPTION

AUSF / UDM

UE 認證流程與訂閱資料查詢

POLICY & CHARGING

PCF / CHF

提供策略決策與計費數據回饋

NETWORK SLICING

NSSF

協助選擇適當的網路切片與路徑

INFRA

NRF / UDR / NEF

服務發現、資料儲存、對外能力開放

➤ NF 與 SBI 服務名稱

NF	SBI 服務前綴	常見用途
AMF	namf-comm	N1/N2 訊息轉送、UE Context
SMF	nsmf-pdusession	建立 / 更新 SM Context
UDM	nudm-sdm / nudm-uecm	查訂閱資料、登記服務中的 AMF
AUSF	nausf-auth	5G AKA / EAP-AKA' 認證
NRF	nnrf-nfm / nnrf-disc	NF 註冊管理、服務發現

規則：n + 小寫 NF 名稱 + 服務名稱，例如 nudm-sdm = UDM 的 Subscriber Data Management。

➤ 控制面 NF VS UPF

控制面 NF

AMF · SMF · NRF · UDM ...

純 Go 程式，跑在 user space

對外提供 SBI (HTTP/2 server)

AMF 另有 SCTP 處理 N2

UPF

go-upf + gtp5g

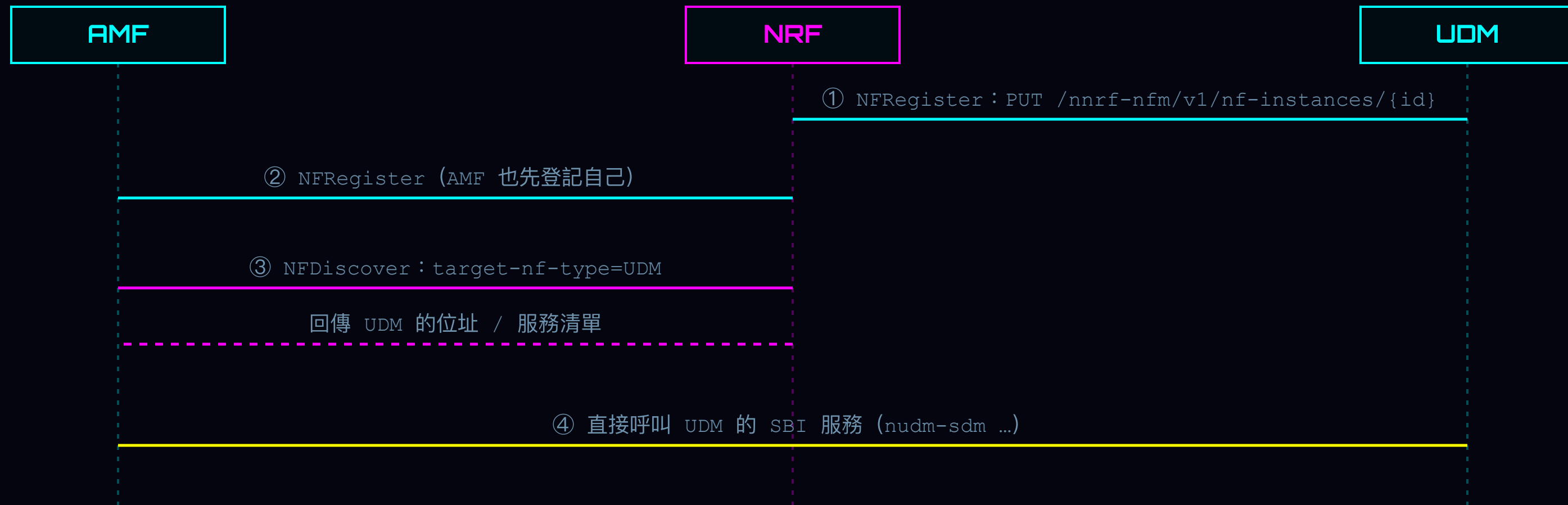
Go 程式負責 N4 PFCP 控制

封包轉送交給 kernel 的 gtp5g

需要 Linux 並載入 gtp5g 模組

CUPS 的精神在程式上也看得到：UPF 只聽 SMF 指令，自己不做決策。

➤ NRF：一切從註冊開始



/> 啟動順序與資料庫

0

MongoDB

UDR、NRF、WebConsole 共用的資料庫，必須先啟動

1

UPF

先備好使用者面，SMF 啟動時才能建立 N4 PFCP Association

2

NRF → AMF / SMF / UDM / ...

控制面 NF：NRF 先起，其餘 NF 再依序啟動並完成 NFRegister

3

WebConsole

用瀏覽器新增訂閱者（IMSI、金鑰、切片、DNN）

\$ PART 04

➤ NF 的程式結構

基於 Release 17 實作，每個 NF 倉庫都長得很像

➤ 典型 NF 倉庫目錄

```
.
├── cmd
│   └── main.go
├── go.mod / go.sum
├── internal
│   ├── context
│   ├── logger
│   ├── sbi
│   │   ├── api_xxx.go
│   │   ├── consumer
│   │   ├── processor
│   │   ├── router.go
│   │   └── server.go
│   └── util
├── LICENSE
└── pkg
```

cmd：程式入口

internal：NF 本體，外部不可 import

sbi：與其他 NF 溝通

pkg：工廠函數、設定載入

➤ CMD/MAIN.GO：一切的起點

```
// 簡化示意
func action(c *cli.Context) error {
    cfg, _ := factory.ReadConfig(path)
    logger.Init(cfg.Logger)
    app, _ := service.NewApp(ctx, cfg)
    app.Start()    // 啟動 SBI server
    return nil
}
```

讀取 YAML 設定

建立 context

初始化日誌

啟動對外介面

go.mod / go.sum：記錄依賴的外部套件與版本（例如 free5gc/openapi），確保大家建置結果一致。

/> INTERNAL/CONTEXT 與 LOGGER

context

執行上下文

NF 的全域狀態，例如 AMF 的 UE 列表

NF 自身資訊：ID、SBI 位址

實際的業務邏輯與狀態更新

logger

日誌封裝

統一 log level 與輸出格式

依模組分類：MainLog、GmmLog...

除錯時最先打開的地方

/> INTERNAL/SBI 四件套

API_XXX.GO

服務介面

對應 3GPP 定義的 API，handler 雛形，解析 HTTP 請求

PROCESSOR

商業邏輯

把 request 轉成內部流程：更新狀態機、呼叫 context

CONSUMER

客戶端

主動呼叫其他 NF，如向 UDM 查訂閱、向 PCF 拿策略

ROUTER / SERVER

HTTP 伺服器

把 URL path、method 綁到 handler，啟動 SBI server

／> 每個 NF 都有兩種身分

Producer

被呼叫：server + api + processor

對外開放自己的服務

例：UDM 提供 nudm-sdm

收到請求 → 處理 → 回應

Consumer

去呼叫：consumer/

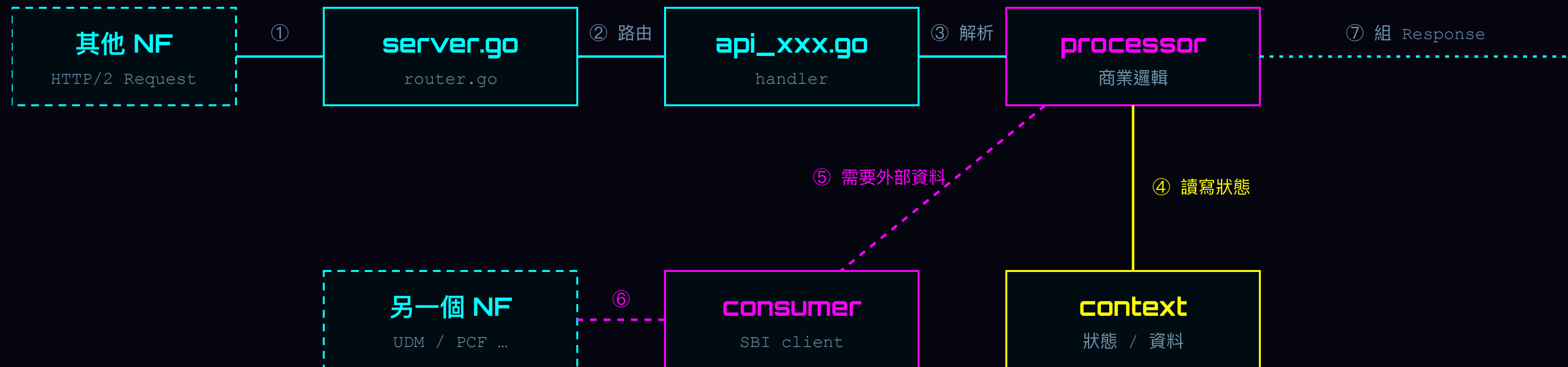
使用其他 NF 的服務

例：AMF 呼叫 UDM 查訂閱

先問 NRF，再發 HTTP 請求

同一個 AMF，對 SMF 是 Producer，對 UDM 是 Consumer：SBA 是一張網，不是一條鏈。

➤ 一個 SBI 請求的旅程



／> 實例：AMF 向 UDM 查訂閱資料

```
GET /nudm-sdm/v2/{supi}/am-data  
Host: <UDM 的 SBI 位址> # 由 NRF 查詢得到
```

AMF：processor 處理註冊時，透過 consumer 發出請求

UDM：router 對到 api handler，交給 processor 處理

UDM 再當 consumer，向 UDR 取出真正的訂閱資料

/> UTIL、PKG 與 LICENSE

util

`internal/util`

字串處理、錯誤包裝

共同常數

避免散落各檔

pkg

`factory / service`

設定檔結構與載入

App 初始化工廠函數

可被外部 import

LICENSE

`Apache 2.0`

允許修改與再散布

需保留版權聲明

研究與產品化都友善

➤ 讀任何一個 NF 的路線圖

1

cmd/main.go

從哪裡啟動？讀了哪些設定？

2

internal/context · **logger**

這個 NF 記住了什麼狀態？log 在哪裡印？

3

internal/sbi

server → api → processor 收請求；consumer 送請求

4

util · **pkg**

哪些工具與共用程式碼可以直接複用？

➤ 本章小結

free5GC：以教學、研究、PoC 為主的 Go 語言開源 5GC

主倉庫 + 各 NF 倉庫 + 共用函式庫，組成完整生態系

NF 依 3GPP 分類，透過 NRF 註冊 / 發現，以 SBI 互相呼叫

NF 程式結構一致：cmd → internal (context / sbi) → pkg

下一章：深入 AMF，從功能角色 → 訊號流程 → 程式實作